ROBOTS MÓVILES AUTÓNOMOS

Ingeniería Matemática e Inteligencia Artificial

Práctica 4: Planificación y Seguimiento de Rutas

Informe de Laboratorio

Autores:
Liam Esgueva González
Jorge Carnicero Príncipe
Andrés Gil Vicente

Fecha de entrega:
XX de XXXX de 2026

Índice

1. Aquí sí, aquí no **3**

2. Conectando los puntos **5**

3. ¿Cuándo llegamos? ¿Falta mucho? **6**

3.1. Diseño del algoritmo A* 6

4. Debes encontrar el camino **9**

5. El rally de Montecarlo **10**

6. Me encanta que los planes salgan bien **12**

6.1. Publicador de la ruta 12

6.2. Publicación del mensaje Path 12

7. Análisis de sensibilidad del PRM **14**

7.0.1. Estrategia de inicialización: rejilla (`use_grid = True`) 14

7.0.2. Estrategia de inicialización: aleatoria (`use_grid = False`) 15

7.0.3. Parámetros de suavizado 17

7.0.4. Configuración final: Compromiso para evitar colisiones 19

8. ¿Quién conoce la ruta? **20**

9. Encontrando el rastro **21**

10. ¡Tierra a la vista! **22**

11. ¡Que empiece la carrera! **24**

12. Sensibilidad de la persecución pura **26**

13. Azul, rosa, azul, rosa... **27**

14. Al andar se hace camino **28**

14.1. Gestión de las paradas del filtro de partículas 28
14.2. Predicción de la pose entre correcciones 29
14.3. Resultados 30

15. Uso auxiliar de Inteligencia Artificial **31**

1. Aquí sí, aquí no

En esta primera sección implementamos la generación de los nodos que conformarán la hoja de ruta probabilística (PRM). El objetivo es completar el método `_generate_nodes` en el archivo `prm.py`, que debe crear puntos de paso válidos dentro del espacio libre del mapa con los obstáculos engrosados.

El método admite dos estrategias de generación de nodos controladas por el argumento `use_grid`:

- **Muestreo aleatorio (PRM clásico):** Cuando `use_grid` es `False`, se generan `node_count` nodos muestreando coordenadas (x, y) de una distribución uniforme dentro de los límites del mapa. Cada nodo candidato se valida con el método `contains` de la clase `Map`, que ya tiene en cuenta el engrosamiento de los obstáculos para garantizar que el robot no colisione al intentar seguir la ruta planificada.
- **Rejilla determinista:** Cuando `use_grid` es `True`, se genera una rejilla regular con paso `grid_size` en ambas direcciones. Todos los puntos de la rejilla que caen en espacio libre se incorporan al grafo.

Código 1.0: Generación de nodos para la hoja de ruta - TODO 4.1

```python
def _generate_nodes(
  self, use_grid: bool = False, node_count: int = 50, grid_size=0.1
) -> dict[tuple[float, float], list[tuple[float, float]]]:

  graph: dict[tuple[float, float], list[tuple[float, float]]] = {}

  # TODO: 4.1. Complete the missing function body with your code.
  x_min, y_min, x_max, y_max = self._map.bounds()

  if use_grid:
      grid = np.mgrid[
          x_min : x_max + grid_size : grid_size,
          y_min : y_max + grid_size : grid_size,
      ]
      for x in grid[0].flat:
          for y in grid[1].flat:
              if self._map.contains((x, y)):
                  graph[(x, y)] = []
  else:
      for _ in range(node_count):
          valid = False
          while not valid:
              particle_x = np.random.uniform(low=x_min, high=x_max)
              particle_y = np.random.uniform(low=y_min, high=y_max)
```

```
            if self._map.contains((particle_x, particle_y)):
                valid = True
                graph[(particle_x, particle_y)] = []

    return graph
```

En el caso aleatorio, utilizamos un bucle `while` de rechazo: generamos coordenadas uniformemente hasta encontrar un punto en espacio libre. Este enfoque es análogo al que utilizamos en el filtro de partículas de la práctica anterior para inicializar las partículas. En el caso de la rejilla, empleamos `np.mgrid` para generar todas las coordenadas de forma eficiente y filtramos únicamente las que se encuentran en espacio libre.

2. Conectando los puntos

Una vez generados los nodos, el siguiente paso es conectarlos para formar un grafo navegable. El método `_connect_nodes` une cada nodo con todos aquellos que están a una distancia menor o igual a `connection_distance` y cuya conexión no atraviese ningún obstáculo.

Esta estrategia basada en un umbral de distancia es preferible a usar los k vecinos más próximos, ya que reduce la probabilidad de que queden subgrafos aislados

Código 2.0: Conexión de nodos del grafo - TODO 4.2

```python
def _connect_nodes(
  self,
  graph: dict[tuple[float, float], list[tuple[float, float]]],
  connection_distance: float = 0.15,
) -> dict[tuple[float, float], list[tuple[float, float]]]:

  # TODO: 4.2. Complete the missing function body with your code.
  for node in graph.keys():
      for other_node in graph.keys():
          if node != other_node:
              distance = np.linalg.norm(
                  np.array(node) - np.array(other_node)
              )
              if (
                  distance <= connection_distance
                  and not self._map.crosses([node, other_node])
              ):
                  graph[node].append(other_node)

  return graph
```

Para cada par de nodos distintos, calculamos la distancia euclídea entre ellos. Si esta distancia no supera el umbral de conexión y la línea recta entre ambos no cruza ningún obstáculo, añadimos el nodo destino a la lista de vecinos del nodo origen.

3. ¿Cuándo llegamos? ¿Falta mucho?

Nuestra implementación sigue los pasos descritos en el enunciado.

3.1. Diseño del algoritmo A*

El algoritmo comienza localizando los nodos del grafo más próximos a los puntos inicial y final proporcionados. A continuación, utiliza una lista abierta (implementada como diccionario con estructura `{(x, y): (f, g)}`) y una lista cerrada (implementada como conjunto de tuplas `(x, y)`). La función heurística h es la distancia euclídea de cada nodo al objetivo, lo que garantiza que la heurística sea admisible y consistente.

En cada iteración se extrae el nodo con menor valor $f = g + h$ de la lista abierta, se expanden sus vecinos y se actualizan los costes si se encuentra una ruta más corta. El diccionario de ancestros permite reconstruir el camino una vez se alcanza el objetivo.

Código 3.0: Implementación del algoritmo A* - TODO 4.3

```python
def find_path(
  self, start: tuple[float, float], goal: tuple[float, float]
) -> list[tuple[float, float]]:

  # Check if the goal is valid
  if not self._map.contains(goal):
      raise ValueError("Goal location is outside the environment.")

  h = {
      node: np.linalg.norm(np.array(node) - np.array(goal))
      for node in self._graph.keys()
  }  # Heuristic function

  ancestors: dict[tuple[float, float], tuple[float, float]] = {}

  # TODO: 4.3. Complete the function body.
  path: list[tuple[float, float]] = []

  # Find the closest nodes in the graph to the start and goal locations
  closest_start_node = min(
      self._graph.keys(),
      key=lambda node: np.linalg.norm(np.array(node) - np.array(start)),
  )
  closest_goal_node = min(
      self._graph.keys(),
      key=lambda node: np.linalg.norm(np.array(node) - np.array(goal)),
  )
```

```
    ancestors[goal] = closest_goal_node
    ancestors[closest_start_node] = start

    g = 0
    f = h[closest_start_node] + g
    open_list = {closest_start_node: (f, g)}
    closed_list = set()

    while len(open_list.keys()) != 0:
        current_node = min(
            open_list.keys(), key=lambda node: open_list[node][0]
        )

        current_g = open_list[current_node][1]

        if current_node == closest_goal_node:
            path = self._reconstruct_path(start, goal, ancestors)
            return path

        closed_list.add(current_node)
        del open_list[current_node]

        neighbors = self._graph[current_node]
        for neighbor in neighbors:
            if neighbor in closed_list:
                continue

            neigh_g = current_g + np.linalg.norm(
                np.array(current_node) - np.array(neighbor)
            )
            if neighbor not in open_list or neigh_g < open_list[neighbor][1]:
                ancestors[neighbor] = current_node
                g = neigh_g
                f = g + h[neighbor]
                open_list[neighbor] = (f, g)

    raise ValueError("No path found from start to goal.")
```

Cabe destacar algunos detalles importantes de nuestra implementación:

- Antes de iniciar la búsqueda, registramos en el diccionario de ancestros las relaciones entre los puntos reales de inicio/fin y los nodos más cercanos del grafo. Esto permite que `_reconstruct_path` genere una ruta que comience y termine exactamente en las coordenadas solicitadas.

- Cuando un vecino ya está en la lista abierta con un coste g mayor, sustituimos su entrada por la nueva ruta más corta y actualizamos el diccionario de ancestros.
- Si la lista abierta se vacía sin haber encontrado el objetivo, lanzamos una excepción indicando que no existe un camino viable.

4. Debes encontrar el camino

Una vez que el algoritmo A* alcanza el nodo objetivo, necesitamos reconstruir el camino completo desde el origen hasta el destino recorriendo el diccionario de ancestros en orden inverso.

Código 4.0: Reconstrucción de la ruta - TODO 4.4

```python
def _reconstruct_path(
  self,
  start: tuple[float, float],
  goal: tuple[float, float],
  ancestors: dict[tuple[int, int], tuple[int, int]],
) -> list[tuple[float, float]]:

  path: list[tuple[float, float]] = []

  # TODO: 4.4. Complete the missing function body with your code.
  current_node = goal
  while current_node != start:
      path.append(current_node)
      current_node = ancestors[current_node]
  path.append(start)
  path.reverse()

  return path
```

El método parte del nodo objetivo y va retrocediendo a través del diccionario de ancestros hasta llegar al nodo de inicio. Cada nodo visitado se añade a la ruta. Finalmente, invertimos la lista para obtener la ruta en el orden correcto, desde el origen hasta el destino.

5. El rally de Montecarlo

Para suavizar esta ruta, aplicamos descenso del gradiente con dos términos ponderados: uno que mantiene la fidelidad al camino original y otro que promueve la suavidad.

El algoritmo utiliza la siguiente regla de actualización iterativa para cada punto intermedio s_i del camino:

$$s_i \leftarrow s_i + w_d \cdot (p_i - s_i) + w_s \cdot (s_{i+1} + s_{i-1} - 2s_i) \tag{1}$$

donde p_i es el punto original, w_d es el peso de fidelidad a los datos y w_s es el peso de suavizado. Los puntos inicial y final se mantienen fijos.

Código 5.0: Suavizado de la ruta por descenso del gradiente - TODO 4.5

```
@staticmethod
def smooth_path(
  path: list[tuple[float, float]],
  data_weight: float = 0.1,
  smooth_weight: float = 0.3,
  additional_smoothing_points: int = 0,
  tolerance: float = 1e-6,
) -> list[tuple[float, float]]:

  # TODO: 4.5. Complete the function body.

  ### Add additional points
  if additional_smoothing_points > 0:
      new_path = []
      for i in range(len(path) - 1):
          p_i = path[i]
          p_next = path[i + 1]

          new_path.append(p_i)

          vector = np.array(p_next) - np.array(p_i)

          for j in range(1, additional_smoothing_points + 1):
              intermediate_point = (
                  p_i[0] + vector[0] * j / (additional_smoothing_points + 1)
    ,
                  p_i[1] + vector[1] * j / (additional_smoothing_points + 1)
    ,
              )
              new_path.append(intermediate_point)
```

```
        new_path.append(path[-1])
        path = new_path

    smoothed_path: list[tuple[float, float]] = []
    s = np.array(path)

    while True:
        s_anterior = np.copy(s)
        smoothed_path: list[tuple[float, float]] = [s[0]]
        for i in range(1, len(path) - 1):
            p_i = path[i]

            s[i] = (
                s[i]
                + data_weight * (np.array(p_i) - np.array(s[i]))
                + smooth_weight * (s[i + 1] + s[i - 1] - 2 * s[i])
            )
            smoothed_path.append(s[i])

        if np.sum(np.abs(s_anterior - s)) < tolerance:
            smoothed_path.append(s[-1])
            return smoothed_path
```

La primera parte del método se encarga de interpolar puntos adicionales entre cada par de nodos de la ruta original. Para ello, calculamos el vector entre nodos consecutivos y distribuimos uniformemente los puntos intermedios a lo largo de dicho segmento.

La segunda parte implementa el descenso del gradiente iterativo. En cada iteración, actualizamos cada punto intermedio con la fórmula descrita anteriormente. El término $w_d \cdot (p_i - s_i)$ tira del punto suavizado hacia su posición original, mientras que $w_s \cdot (s_{i+1} + s_{i-1} - 2s_i)$ tira hacia el punto medio de sus vecinos, suavizando la curva. El proceso se repite hasta que la variación total cae por debajo de la tolerancia.

6. Me encanta que los planes salgan bien

Para transmitir la ruta suavizada al nodo de persecución pura, publicamos un topic denominado `/path` utilizando el mensaje `nav_msgs/msg/Path`.

6.1. Publicador de la ruta

Creamos el publicador en el método `on_configure` del nodo PRM:

Código 6.0: Creación del publicador de la ruta - TODO 4.6

```
# Publishers
# TODO: 4.6. Create the /path publisher (Path message).
self._path_publisher = self.create_publisher(Path, "/path", qos_profile=10)
```

6.2. Publicación del mensaje Path

La función `_publish_path` convierte la lista de tuplas (x, y) que conforma la ruta suavizada en un mensaje de tipo `Path`. Cada punto del camino se encapsula en un `PoseStamped` con su correspondiente cabecera temporal:

Código 6.1: Publicación de la ruta suavizada - TODO 4.7

```
def _publish_path(self, path: list[tuple[float, float]]) -> None:

  # TODO: 4.7. Complete the function body.
  msg = Path()
  msg.header.stamp = self.get_clock().now().to_msg()
  msg.header.frame_id = "map"

  poses_list = []

  for point in path:
      pose_msg = PoseStamped()
      pose_msg.header = msg.header

      pose_msg.pose.position.x = float(point[0])
      pose_msg.pose.position.y = float(point[1])
      pose_msg.pose.position.z = 0.0

      poses_list.append(pose_msg)

  msg.poses = poses_list
```

```
self._path_publisher.publish(msg)
```

Asignamos el `frame_id` como `"map"` para indicar que las coordenadas están referidas al sistema de referencia del mapa. Cada punto de la ruta se convierte en un `PoseStamped`, donde solo rellenamos las componentes x e y de la posición, dejando $z = 0$ ya que el robot se mueve en el plano.

7. Análisis de sensibilidad del PRM

Pregunta: Realiza experimentos para escoger valores razonables para los hiperparámetros del PRM y evalúa su impacto en los tiempos de cálculo.

Durante la implementación de la hoja de ruta probabilística nos encontramos con múltiples hiperparámetros que afectan tanto a la calidad de la ruta planificada como al tiempo de cálculo. Antes de analizar cada uno por separado, conviene destacar una restricción fundamental: existe una dependencia crítica entre `node_count` y `connection_distance`. Un número elevado de nodos con una distancia de conexión demasiado pequeña puede impedir encontrar un camino, ya que los nodos quedan aislados; y a la inversa, una distancia de conexión demasiado grande con pocos nodos aumenta el riesgo de que las conexiones crucen obstáculos y sean descartadas.

7.0.1. Estrategia de inicialización: rejilla (`use_grid = True`)

En la estrategia de rejilla, los hiperparámetros más relevantes son `grid_size` y `connection_distance`. A continuación comparamos dos configuraciones.

Configuración 1: `grid_size = 0.1 m`, `connection_distance = 0.20 m`

Con un paso de rejilla fino de 0.1 m se genera un gran número de nodos, lo que resulta en un roadmap denso y bien conectado capaz de encontrar un camino correcto. Sin embargo, el tiempo de creación del roadmap es elevado debido al alto número de pares de nodos a evaluar. (reducir la rejilla a 0.05 m dispara el tiempo a más de 170 segundos, descartándolo para uso real).

- Roadmap creation time: 13.270 s Pathfinding time: 0.006 s Smoothing time: 0.014 s

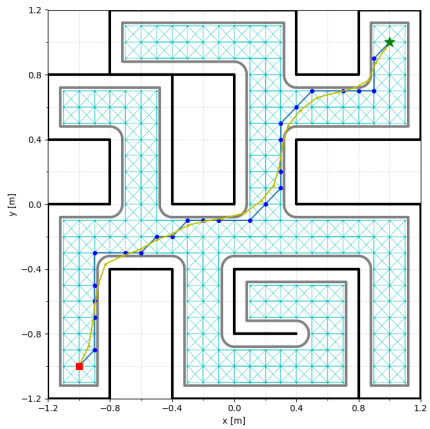

Figura 1: Hoja de ruta con `use_grid=True`, `grid_size=0.1`, `connection_distance=0.20`.

Configuración 2: `grid_size = 0.2 m`, `connection_distance = 0.30 m`

Con un paso de rejilla de 0.2 m se generan muchos menos nodos, lo que reduce drásticamente el tiempo de creación del roadmap. No obstante, es necesario aumentar `connection_distance` a 0.30 m, ya que con 0.20 m los nodos (diagonales) pueden no llegar a conectarse / no se conectaran. La desventaja de esta configuración es que la calidad de la ruta depende de la regularidad del mapa.

- Roadmap creation time: 1.425 s Pathfinding time: 0.001 s Smoothing time: 0.008 s

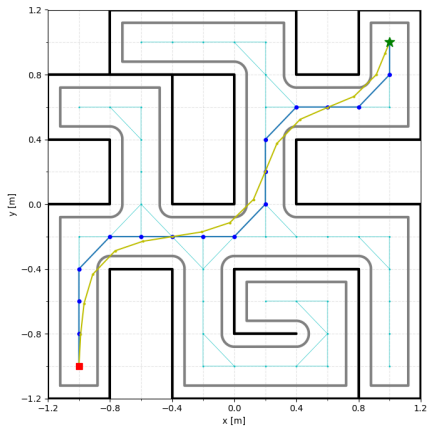

Figura 2: Hoja de ruta con `use_grid=True`, `grid_size=0.2`, `connection_distance=0.30`.

7.0.2. Estrategia de inicialización: aleatoria (`use_grid = False`)

En la estrategia aleatoria, los parámetros principales son `node_count` y `connection_distance`.

Configuración 1: `node_count = 1000`, `connection_distance = 0.20 m`

Con 1000 nodos y una distancia de conexión moderada se obtiene un roadmap denso que encuentra el camino de forma fiable, con un tiempo de creación razonable. Esta se podría ser una de las configuraciones más equilibradas.

- Roadmap creation time: 5.428 s Pathfinding time: 0.039 s Smoothing time: 0.013 s

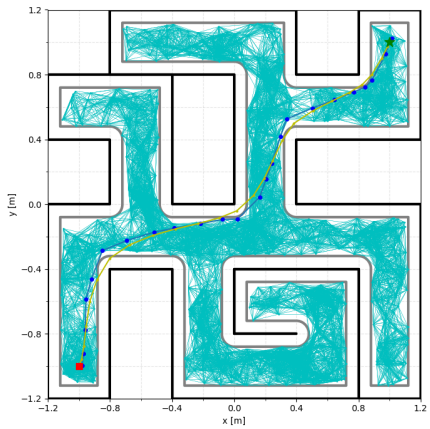

Figura 3: Hoja de ruta con `use_grid=False`, `node_count=1000`, `connection_distance=0.20`.

Configuración 2: `node_count = 1000`, `connection_distance = 0.10 m`

Reducir la distancia de conexión disminuye ligeramente el tiempo de creación, ya que cada nodo se conecta con menos vecinos. Sin embargo, aumenta peligrosamente el riesgo de que aparezcan subgrafos aislados y caminos sin salida, especialmente en zonas más estrechas.

- Roadmap creation time: 4.335 s Pathfinding time: 0.020 s Smoothing time: 0.028 s

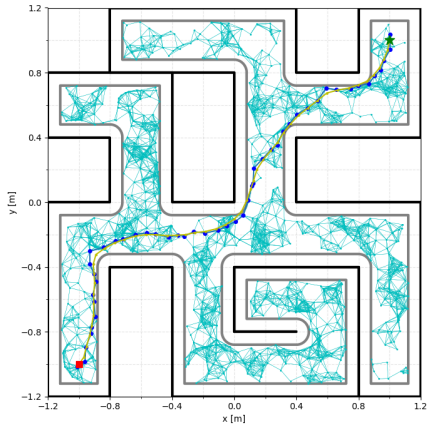

Figura 4: Hoja de ruta con `use_grid=False`, `node_count=1000`, `connection_distance=0.10`. Se aprecian caminos muertos.

Configuración 3: `node_count = 10000`, `connection_distance = 0.05 m`

Para ilustrar un caso extremo, si se busca conectividad con una distancia de conexión muy pequeña (0.05 m), es necesario recurrir a un número de nodos exageradamente alto (10000). El resultado es un tiempo de creación del roadmap completamente inviable (casi 5 minutos).

- Roadmap creation time: 295.403 s Pathfinding time: 0.379 s Smoothing time: 0.046 s

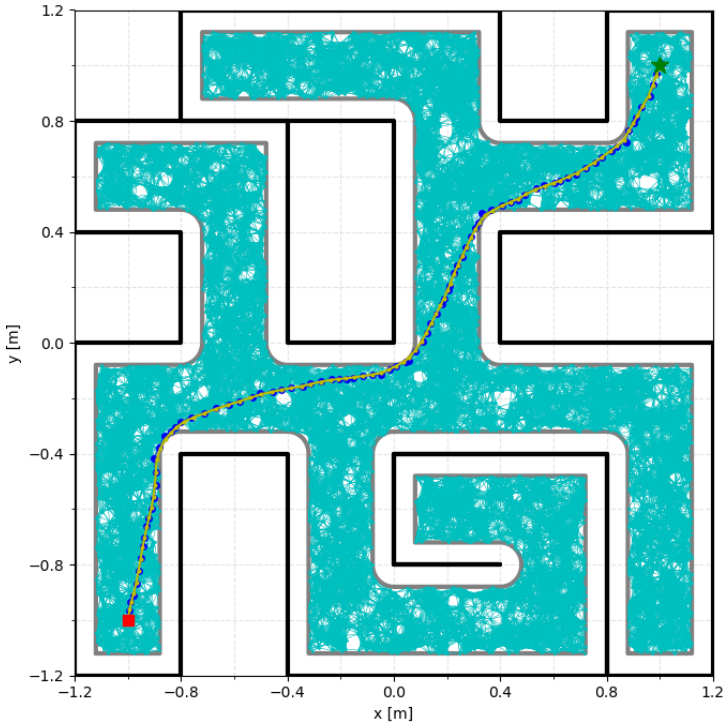

Figura 5: Hoja de ruta con `use_grid=False`, `node_count=10000`, `connection_distance=0.05`.

7.0.3. Parámetros de suavizado

Para el análisis del suavizado fijamos la configuración del grafo en `use_grid=False`, `node_count=500`, `connection_distance=0.20`, centrándonos en el efecto visual y temporal de los pesos `data_weight` y `smooth_weight`, manteniendo `additional_smoothing_points = 3`.

Configuración base: `data_weight = 0.1`, `smooth_weight = 0.1`

Con pesos bajos y equilibrados se obtiene una ruta moderadamente suavizada. El tiempo de suavizado es despreciable frente al de creación del roadmap.

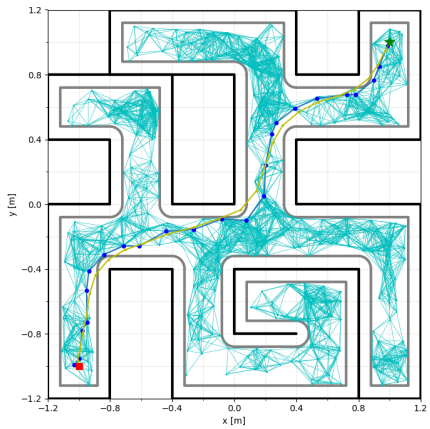

Figura 6: Suavizado con `data_weight=0.1`, `smooth_weight=0.1`.

Caso límite (sin suavizado): `data_weight = 1`, `smooth_weight = 0`

Al anular el peso de suavizado, la ruta de salida coincide exactamente con la ruta original del grafo, como vimos en la teoría.

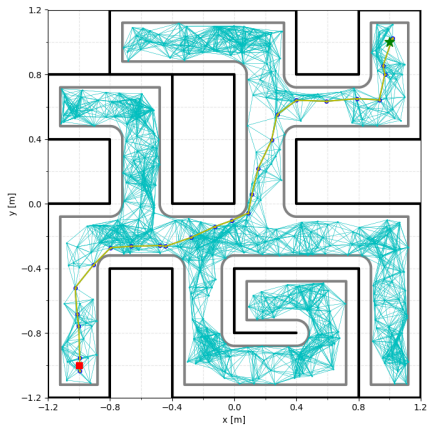

Figura 7: Sin suavizado efectivo: `data_weight=1`, `smooth_weight=0`.

Alta fidelidad con suavizado moderado: `data_weight = 0.9`, `smooth_weight = 0.5`

Esta configuración prioriza fuertemente la fidelidad al camino original mientras aplica un suavizado que recorta los vértices más agresivos. Es la que mejor equilibra precisión y navegabilidad, manteniéndose alejada de los obstáculos.

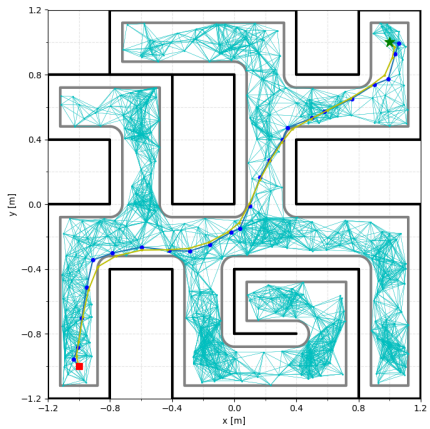

Figura 8: Configuración seleccionada: `data_weight=0.9`, `smooth_weight=0.5`.

La principal conclusión aquí es que resulta más eficiente mantener un número de nodos moderado en el grafo y delegar la calidad de la trayectoria final al algoritmo de suavizado mediante la adición de puntos intermedios.

7.0.4. Configuración final: Compromiso para evitar colisiones

Para que el robot siga la ruta planificada sin colisionar, es crítico equilibrar el suavizado de la trayectoria con el engrosamiento de los obstáculos. Un suavizado excesivo 'recorta' las esquinas y acerca el robot a las paredes, pero si compensamos esto engrosando demasiado los obstáculos, corremos el riesgo de que sean demasiado cerrados los pasillos y que no haya conexiones en estos.

Tras realizar diversos experimentos, hemos llegado a una configuración óptima:

- **Engrosamiento de obstáculos:** Lo hemos aumentado ligeramente respecto al valor predeterminado. Esto nos da un margen de seguridad vital en las esquinas sin llegar a bloquear los caminos viables.
- **Generación del grafo:** `node_count = 750` y `connection_distance = 0.2`. Esta combinación asegura una conectividad larga y robusta sin exceder el coste computacional.
- **Suavizado:** `additional_smoothing_points = 5`, con `data_weight = 0.5` y `smooth_weight = 0.2`. Aplicamos un suavizado moderado que no deforma en exceso la ruta original, dando un mayor peso a ser fíel a la ruta original pero sin dejar de suavizar la ruta.

Ventaja adicional: Al inyectar 5 puntos intermedios entre cada par de nodos consecutivos, obtenemos una trayectoria mucho más densa. Esto facilita y acelera enormemente el trabajo del controlador posterior (Pure Pursuit), ya que la búsqueda del punto objetivo a la distancia de anticipación (*lookahead distance*) resulta mucho más rápida y precisa.

8. ¿Quién conoce la ruta?

Implementamos el callback `_path_callback` que procesa el mensaje `Path` y almacena las coordenadas en la clase `PurePursuit` mediante su propiedad `path`.

Código 8.0: Recepción de la ruta en el nodo de persecución pura - TODO 4.8

```
def _path_callback(self, path_msg: Path):
  # TODO: 4.8. Complete the function body.
  path_coordinates = [
      (pose.pose.position.x, pose.pose.position.y)
      for pose in path_msg.poses
  ]

  self._pure_pursuit.path = path_coordinates
```

La propiedad `path` del objeto `PurePursuit` se encarga de guardar internamente esta información para su uso posterior en el cálculo de los comandos de velocidad.

9. Encontrando el rastro

En lugar de trabajar con la pose del robot respecto al mapa, ahora necesitamos determinar su localización respecto a la ruta. El método `_find_closest_point` obtiene el punto de la ruta más próximo al robot.

Código 9.0: Punto más cercano de la ruta - TODO 4.9

```python
def _find_closest_point(
  self, x: float, y: float
) -> tuple[tuple[float, float], int]:

  # TODO: 4.9. Complete the function body.
  path = np.array(self._path)

  dx = path[:, 0] - x
  dy = path[:, 1] - y
  squared_distances = dx**2 + dy**2
  closest_idx = int(np.argmin(squared_distances))
  closest_xy = (path[closest_idx][0], path[closest_idx][1])

  return closest_xy, closest_idx
```

Utilizamos operaciones vectorizadas de NumPy para mejorar el rendimiento.

10. ¡Tierra a la vista!

El método `_find_target_point` determina el punto de la ruta hacia el que el robot debe dirigirse, situado a una distancia de anticipación (*lookahead*) desde la posición actual del robot.

Código 10.0: Punto objetivo a la distancia de anticipación - TODO 4.10

```python
def _find_target_point(
  self, origin_xy: tuple[float, float], origin_idx: int
) -> tuple[float, float]:

  # TODO: 4.10. Complete the function body.
  path = np.array(self._path)
  np_origin_xy = np.array(origin_xy)

  future_path = path[origin_idx:]

  if len(future_path) == 0:
      return tuple(path[-1])

  dx = future_path[:, 0] - origin_xy[0]
  dy = future_path[:, 1] - origin_xy[1]
  squared_distances = dx**2 + dy**2

  lookhead_sqr = self._lookahead_distance**2
  idxs = np.where(squared_distances >= lookhead_sqr)[0]

  if len(idxs) > 0:
      return tuple(future_path[idxs[0]])

  return tuple(path[-1])
```

Nuestra estrategia para gestionar el caso en el que no hay un punto exactamente a la distancia de anticipación es la siguiente:

- Recortamos la ruta desde el punto más cercano en adelante (`future_path`), descartando la porción ya recorrida.
- Calculamos las distancias al cuadrado de todos los puntos futuros respecto a la posición actual del robot.
- Seleccionamos el primer punto cuya distancia es igual o superior a la distancia de anticipación. Esta decisión conservadora garantiza que el robot siempre mire lo suficientemente lejos hacia adelante.

- Si ningún punto de la ruta restante se encuentra a la distancia de anticipación, devolvemos el último punto del camino, lo que indica que el robot está cerca del destino y debe dirigirse directamente hacia él.

Hemos optado por esta aproximación discreta en lugar de la interpolación lineal entre puntos o una media entre el anterior y el siguiente, ya que dado que nuestra ruta suavizada contiene puntos intermedios adicionales, la resolución es suficiente para obtener un punto objetivo razonablemente preciso sin el coste adicional de la interpolación, de forma que ahorramos en computo.

11. ¡Que empiece la carrera!

La persecución pura calcula en cada instante las velocidades lineal v y angular ω que el robot debe aplicar para seguir la ruta.

A partir de la fórmula de la persecución pura deducida en clase:

$$\omega = \frac{2v\sin(\alpha)}{l} \tag{2}$$

donde α es el ángulo entre la orientación del robot y la dirección al punto objetivo, y l es la distancia al punto objetivo.

Además, hemos incorporado una velocidad adaptativa: cuando el ángulo α es grande, reducimos la velocidad lineal para mejorar la estabilidad en las curvas. Si el ángulo supera un umbral máximo ($\pi/3{,}5$ radianes), el robot gira sobre sí mismo sin avanzar, evitando describir circunferencias amplias que podrían provocar chocarse.

Código 11.0: Controlador de persecución pura - TODO 4.11

```python
def compute_commands(
  self, x: float, y: float, theta: float
) -> tuple[float, float]:

  # TODO: 4.11. Complete the function body.

  if not self._path or len(self._path) == 0:
      return 0.0, 0.0

  L = self._find_target_point(
      (x, y), self._find_closest_point(x, y)[1]
  )

  vector_to_target = np.array(L) - np.array((x, y))
  l = np.linalg.norm(vector_to_target)

  raw_alpha = np.arctan2(vector_to_target[1], vector_to_target[0]) - theta
  alpha = (raw_alpha + np.pi) % (2 * np.pi) - np.pi

  # Configuration parameters
  v_min = 0.10
  v_max = 0.22
  max_angle = np.pi / 3.5

  if abs(alpha) > max_angle:
      w = 0.5 * np.sign(alpha)
      return 0.0, w
```

```
    # Adaptive velocity: reduce speed as steering angle increases
    v_desired = v_max * (1 - abs(alpha) / max_angle)
    v = max(v_min, min(v_desired, v_max))

    w = (
        v * 2 * np.sin(alpha) / l
        if l > 0
        else 0.0
    )

    return v, w
```

Detallamos las decisiones de diseño más relevantes:

- **Protección ante ausencia de ruta:** Si no se ha recibido aún el camino del nodo de planificación, el robot publica velocidades nulas para mantener el flujo de información entre los demás nodos de ROS sin moverse.
- **Normalización del ángulo:** Aplicamos la operación $(\alpha + \pi) \mod 2\pi - \pi$ para garantizar que el ángulo de dirección siempre esté en el rango $[-\pi, \pi]$, evitando giros innecesarios de más de 180°.
- **Giro sobre sí mismo:** Cuando $|\alpha| > \pi/3{,}5$ radianes, el robot gira en el sitio a $\pm 0{,}5$ rad/s en la dirección adecuada. Esto es posible gracias a la cinemática diferencial del TurtleBot3, y es clave para evitar que la persecución pura intente describir una circunferencia amplia que podría colisionar con paredes.
- **Velocidad adaptativa:** La velocidad lineal se reduce linealmente conforme aumenta el ángulo de dirección, con un mínimo de 0.10 m/s y un máximo de 0.22 m/s. Esto garantiza que el robot tome las curvas de forma segura, reduciendo la velocidad cuando necesita girar más, como solemos hacer cuando conducimos nosotros.

12. Sensibilidad de la persecución pura

Pregunta: Evalúe el efecto de modificar la distancia de anticipación y su relación, si es que existe, con la velocidad lineal del robot.

La distancia de anticipación (*lookahead distance*) es el hiperparámetro más crítico del controlador de persecución pura. Determina a qué distancia por delante de su posición actual el robot mira para encontrar el punto objetivo.

- **Distancia de anticipación pequeña** ($l \approx 0{,}15$ m): El robot sigue la ruta de forma muy precisa pero tiende a oscilar, ya que responde de forma agresiva a pequeñas desviaciones. En curvas cerradas puede funcionar bien, pero en rectas genera un comportamiento serpentino.
- **Distancia de anticipación grande** ($l \approx 0{,}5$ m): El robot sigue una trayectoria más suave, cortando las curvas. Esto reduce las oscilaciones pero puede hacer que el robot se desvíe significativamente de la ruta planificada, especialmente en pasillos estrechos.
- **Valor seleccionado** ($l = 0{,}3$ m): Encontramos que este valor ofrece un buen compromiso entre precisión de seguimiento y suavidad del movimiento para nuestro entorno.

Respecto a la relación con la velocidad lineal, observamos que existe una dependencia directa: a mayor velocidad, se necesita una distancia de anticipación proporcionalmente mayor para mantener un seguimiento estable. Esto se debe a que a mayor velocidad, el robot recorre más distancia entre ciclos de control, por lo que necesita mirar más lejos para poder reaccionar a tiempo ante los cambios de dirección de la ruta.

13. Azul, rosa, azul, rosa...

En esta práctica, el robot parte de una pose conocida, por lo que no es necesario ejecutar el nodo del seguidor de pared. Sin embargo, en el proyecto final el robot deberá localizarse primero (utilizando el seguidor de pared y el filtro de partículas) antes de poder planificar y seguir una ruta. Para prevenir que el robot reciba órdenes contradictorias del seguidor de pared y de la persecución pura simultáneamente, añadimos una suscripción condicional al topic `/pose` en la sincronización de mensajes del nodo `wall_follower`.

Código 13.0: Suscripción condicional a /pose - TODO 4.12

```
# TODO: 4.12. Add /pose to the synced subscriptions only if localization is
    enabled.
if enable_localization:
  self._subscribers.append(
      message_filters.Subscriber(
          self, PoseStamped, "pose", qos_profile=10
      )
  )
```

Esta modificación solo añade el topic `/pose` a los suscriptores sincronizados cuando el parámetro `enable_localization` está activado. Gracias a ello, podemos seguir lanzando las prácticas anteriores sin problemas. Cuando la localización está habilitada, el callback `_compute_commands_callback` recibe también el mensaje de pose estimada y, mediante la condición `if not pose_msg.localized`, deja de publicar comandos de velocidad una vez que el robot se ha localizado, cediendo el control a la persecución pura.

14. Al andar se hace camino

Una vez comprobado que el sistema funciona correctamente en simulación, el siguiente paso es transferir el código al TurtleBot3 Burger real. Afortunadamente, la mayor parte del sistema funciona sin modificaciones significativas, a excepción de las limitaciones computacionales del filtro de partículas.

14.1. Gestión de las paradas del filtro de partículas

Las limitaciones computacionales de la Raspberry Pi del TurtleBot3 obligan a detener el robot brevemente mientras el filtro de partículas realiza la fase de corrección. Para coordinar estas paradas, el nodo de persecución pura se suscribe al topic `stop_condition`.

En el callback de esta suscripción, comprobamos si se cumple la condición de parada; de ser así, detenemos el robot publicando comandos de velocidad nulos y guardamos este estado en el atributo `_robot_state_stop`. Además, integramos este atributo en el bucle principal de control para asegurar que no se calculen ni publiquen nuevos comandos de movimiento mientras el filtro de partículas esté ocupado.

Código 14.0: Suscripción al topic de parada y gestión del estado - Apartados 14.1 y 14.2

```python
# En la inicialización del nodo (ej. __init__):
self._robot_state_stop = False
self._subsciber_stop_msg = self.create_subscription(
    ControlStop, "stop_condition", self._stop_msg_callback, 10
)

def _stop_msg_callback(self, stop_msg: ControlStop):
    """Subscriber callback. Stops the robot if a stop condition is met.

    Args:
        stop_msg: Message containing the stop condition.
    """
    if stop_msg.stop_robot:
        self.get_logger().info("Stop condition met. Stopping the robot.")
        self._publish_velocity_commands(0.0, 0.0)

    self._robot_state_stop = stop_msg.stop_robot

# En el método principal de cálculo de comandos:
def _compute_commands_callback(self, ...):
    # Verificación del estado antes de calcular comandos
    if self._robot_state_stop:
        self._publish_velocity_commands(0.0, 0.0)
```

```
        return # Con esto hacemos el 14.2 que no pueda publicar comandos de
    velocidad

    # ... [Resto de la lógica de persecución pura] ...
```

14.2. Predicción de la pose entre correcciones

Para que el nodo de persecución pura pueda seguir ejecutándose fluidamente entre las fases de corrección del filtro de partículas, hemos modificado el nodo del filtro para que publique una predicción de la pose de forma continua.

Cada vez que se recibe un mensaje de odometría, extraemos las velocidades y calculamos el incremento de tiempo real (`dt`) utilizando las marcas temporales de la cabecera del mensaje.

Código 14.1: Predicción de la pose mediante odometría - Apartado 14.3

```
def _callback_odometry_saving_history(self, odom_msg: Odometry):
    """Subscriber callback. Executes a particle filter and publishes (x, y,
    theta) estimates.

    Args:
        odom_msg: Message containing odometry measurements.

    """
    # Parse measurements
    z_v: float = odom_msg.twist.twist.linear.x
    z_w: float = odom_msg.twist.twist.angular.z

    if abs(z_v) >= 1e-6 or abs(z_w) >= 1e-6:
        self._odometry_estimate_list.append((z_v, z_w)) # 3.11.3

    # Dead-reckoning with actual dt from message timestamps
    t = odom_msg.header.stamp.sec + odom_msg.header.stamp.nanosec * 1e-9
    if self._localized and self._prev_odom_time is not None:
        actual_dt = t - self._prev_odom_time
        if 0.0 < actual_dt < 1.0:
            self._predicted_x += z_v * math.cos(self._predicted_theta) *
    actual_dt
            self._predicted_y += z_v * math.sin(self._predicted_theta) *
    actual_dt
            self._predicted_theta += z_w * actual_dt
            self._predicted_theta %= 2 * math.pi
            self._publish_pose_estimate(
                self._predicted_x, self._predicted_y, self._predicted_theta
```

```
            )
    self._prev_odom_time = t
```

14.3. Resultados

Vídeo #1

Este vídeo se adjunta en la entrega junto con el resto de códigos y materiales pedidos. Tiene el nombre de `RobotsMovilesAutonomos_P4_G21_Simulation`.

15. Uso auxiliar de Inteligencia Artificial

A lo largo de la elaboración de esta memoria se ha empleado inteligencia artificial como herramienta de apoyo, sin sustituir en ningún caso el trabajo de comprensión, experimentación y redacción realizado por los autores. En concreto, su uso se ha limitado a tareas auxiliares orientadas a mejorar la claridad y la presentación del documento:

- **Corrección lingüística:** se ha utilizado para detectar y corregir faltas de ortografía, erratas y pequeños problemas de concordancia o puntuación.
- **Mejora del estilo y uniformidad:** se ha empleado para sugerir ajustes de redacción (frases más claras, eliminación de repeticiones y homogeneización del tono).
- **Generación de la estructura del informe:** se utilizó para adaptar la plantilla del informe de la práctica anterior a los requisitos de la práctica actual, manteniendo la coherencia en formato y estilo.